DressMe

Technical report

An integrated fashion recommendation,
wardrobe and virtual try-on platform

BO1–BO7 | Integrated prototype

Project team

Elyes Regaia
Aymen Dhieb
Maissa Khemiri
Syrine Guemira
Neirouz Hannachi
Rihab Ben Hadj Slimen

11 November 2026

Abstract

DressMe integrates outfit and item recommendation, wardrobe analysis, style modelling, a conversational assistant, garment digitization and virtual try-on. The system combines visual and textual embeddings, trained classifiers and compatibility models, explicit wardrobe rules and private user state.

The assistant uses a QLoRA-adapted Qwen3-1.7B parser, hybrid BM25 and multilingual E5 retrieval, bounded outfit assembly and BO1 compatibility scoring. On 80 held-out synthetic requests, 100% of raw parser outputs satisfied the schema and 71.25% matched the full target structure. BO3 achieved an F1 score of 0.918 on its integrated duplicate calibration set. BO6 EfficientNet-B0 reached 0.8882 macro-F1 on 2,310 catalog validation images. The BO7 prototype recorded 36 generated images across three Tesla T4 configurations. The current checkout passed 367 application tests and 72 BO3 tests in separate runs.

The team presentation defines the seven objectives. The repository and recorded experiments establish the behaviour of the integrated application and the scope of the reported measurements.

Project evidence [D1], [D4], [D7], [D10], [D12].

Contents

1 Problem statement 2

2 System architecture and data 3

3 BO1 – Outfit Recommendation 4

4 BO2 – Item Recommendation 6

5 BO3 – Redundancy Detection 7

6 BO4 – Personalized Style Modeling 8

7 BO5 – Explainable Fashion Assistant 9

8 BO6 – Smart Wardrobe Digitization 15

9 BO7 – Personalized Virtual Try-On 16

10 Integrated experiments and results 18

11 Scope and conditions of evaluation 19

12 Reproducing the validation 19

13 Evidence register 20

1 Problem statement

A wardrobe owner needs to combine garments, identify useful additions, recognize duplicates and judge whether a recommendation reflects their taste. Photos and natural-language requests provide convenient inputs, but each supplies only part of the information needed to make a recommendation. DressMe connects these decisions through a shared catalog, signed-in account and private wardrobe.

The technical problem is to carry consistent user context and garment references through several models and algorithms. A candidate can receive a high compatibility score while a required category, ownership filter or occasion rule determines whether it belongs in the pool. The application records the relevant inputs and interprets each signal in the context of its task.

Business objective	**Observable behaviour**
BO1 – Outfit Recommendation	Score a look, suggest compatible swaps, complete a partial look and assemble an outfit for weather and occasion.
BO2 – Item Recommendation	Find visually similar garments and recommend additions that increase wardrobe combinations.
BO3 – Redundancy Detection	Surface same-type duplicates, request confirmation on ambiguous pairs and summarize the wardrobe.
BO4 – Personalized Style Modeling	Infer a style profile and adapt item recommendations to reactions and exploration.
BO5 – Explainable Fashion Assistant	Convert requests into checked plans, compose or revise outfits, and show product and score evidence.
BO6 – Smart Wardrobe Digitization	Suggest fields from a garment photo, collect confirmation and save an editable private record.
BO7 – Personalized Virtual Try-On	Prepare a person image, submit a garment sequence and retain the result in private history.

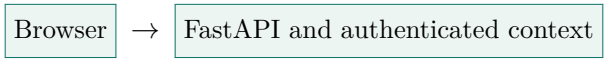
Evaluation design

The evaluation separates predictive quality, ranking behaviour, model reproducibility and application integration. Parser accuracy uses structured target plans. Compatibility and completion use outfit tasks. Duplicate detection and garment classification use their own evaluation datasets. Application tests verify ownership, stable references, score reproduction and job state transitions.

Project evidence [D1] pp. 4–35; [D2].

2 System architecture and data

FastAPI serves the browser application and mounts Studio at `/studio-api`. Uvicorn runs the ASGI application. The local profile uses SQLite and Alembic migrations. Uploaded garments, person images and generated results use private storage. The server supplies account identity and applies session, ownership and CSRF checks where appropriate.



Catalog features	Studio and BO6	BO7 jobs
BO1–BO5 routes, product indices, model exports and chat state.	Accounts, wardrobe review and a separate garment inference service.	Job records, Redis/RQ, worker and local or remote try-on backend.

Figure 1: Application routing and inference responsibilities.

Data view	Size	Use
BO1/BO5 catalog	653 products	Local images, product records and 512-dimensional compatibility embeddings.
BO2 visual search	1,011 vectors	512-dimensional index with garment attributes.
BO3 integrated index	653 vectors	Similarity and colour statistics.
BO4 Taste catalog	8,189 rows	512-dimensional embeddings and style features.
Private wardrobe	Per account	Confirmed records and private image references.

Product codes align records, images and embeddings within each export. Features that cross catalogs use explicit mappings. Studio garments use account-owned UUIDs. The BO5 score contract uses the product codes of its aligned catalog. An optional overlay supplies eligible Studio pieces to wardrobe completion. Users confirm proposed garment attributes before saving, and BO7 records consent for the relevant person input.

Project evidence [D2], [D3], [D5], [D6], [D8].

3 BO1 – Outfit Recommendation

Compatibility scoring and replacement

The scorer projects each 512-dimensional image embedding to 256 dimensions. A learned aggregation token and a two-layer Transformer with four attention heads summarize the outfit. The model masks unused slots and maps its score output to 0–10. A second head predicts an unusual item or a dedicated no-outlier class.

The notebook trained the Transformer on source outfits and artificial changes to their composition. Validation AUC selected the retained checkpoint. The export uses the trained Transformer and aligned CLIP embeddings. The following numbers describe the notebook's recorded evaluation.

Measure	Initial model	Trained Transformer
AUC	0.49	0.73
Outlier identification	0.29	0.37
Replacement top-5	0.23	0.69
MAE on the notebook score target	2.44	2.64

The best recorded validation AUC was 0.82. The integrated Rate My Look view presents an outfit score and possible replacements. It can also use a separate optional model to express the stylist comment, with a template path available.

BO5 interface and score reproduction

The chat sends product codes to `BO1Backend.score`. The backend retrieves the corresponding embedding rows, evaluates sets of up to five distinct items on CPU and returns the exact codes, model identifier and score. BO5 checks that each returned score belongs to its submitted set.

Reference	Reloaded score	Notebook rounding
Three recorded product codes	3.418087	3.42
Four recorded product codes	3.318662	3.32

The two references establish score reproducibility for the same embeddings. The score estimates compatibility for a specified set of garments under the learned model.

Project evidence [D2], [D4], [D5].

Outfit of the Day and Complete My Outfit

Outfit of the Day uses weather, occasion and garment-slot rules to decide which clothing roles to fill. The system compares catalog embeddings with relevant text prompts, builds a short candidate list for each slot and chooses a compatible combination. Eligible catalog pieces can be sent to BO7.

The `/api/ootd` route obtains weather context from a location, city or simulated scenario. It standardizes the similarities within prompt groups, combines them with the occasion term and keeps up to five candidates per role by default. The combination stage evaluates the Cartesian product of the available role lists. For a candidate outfit O, let z_i denote garment i's standardized role score and e_i its normalized catalog embedding. The runtime computes

$$S_{\mathrm{OOTD}}(O) = \frac{1}{|O|}\sum_{i\in O} z_i + \gamma\ \mathrm{zscore}_{\mathrm{pool}}\Big(\mathrm{mean}_{i\neq j}\, e_i^{\top} e_j\Big), \qquad \gamma = 0.5.$$

The configuration can apply an occasion-specific dress adjustment when comparing one-piece and separate top-and-bottom templates. BO5 uses the trained compatibility scorer through its product-code contract.

Complete My Outfit uses a separate trained MiniGPT export and its catalog tables. Given a partial outfit, it predicts a suitable missing item, optionally restricts candidates to a supplied category and re-ranks the results. This model supports the completion screen. The chat uses BO5 assembly and the BO1 scoring contract.

Completion experiment

The notebook records 310 missing-item cases from 92 held-out outfit records and a 259-item evaluation catalog. Providing the missing category constrains the candidate set. The inferred-category mode ranks across the catalog.

Method and mode	Recall@1	Recall@5	Recall@10
Category known, visual CLIP baseline	0.271	0.558	0.742
Category known, GPT	0.516	0.645	0.726
Category known, GPT with re-ranking	0.532	0.671	0.742
Category inferred, visual CLIP baseline	0.023	0.135	0.213
Category inferred, GPT	0.319	0.523	0.539
Category inferred, GPT with re-ranking	0.323	0.526	0.552

The reported mean of Recall@1 and Recall@5 across both validation modes increased from 0.667 to 0.694 with re-ranking. On the recorded fill-in-the-blank task, the model selected the held-out piece among four same-category choices in 0.70 of cases. Random selection gives 0.25 in that setup. Recall measures the rank of the annotated completion.

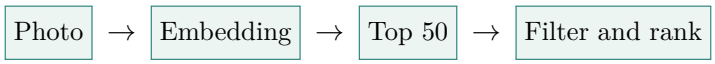
Project evidence [D2], [D4].

4 BO2 – Item Recommendation

Visual Fashion Search

Marqo FashionCLIP encodes the query image. The application computes cosine similarity over 1,011 normalized catalog vectors and retains the 50 closest records. The best matching metadata record supplies type and sleeve references. A SigLIP-based classifier estimates colour.

The re-ranker uses visual similarity (0.60), attribute overlap (0.20) and colour agreement (0.20), with a 0.05 bonus for the same category. It checks compatible category, colour and sleeve, requires attribute overlap and removes very similar results at a cosine threshold of 0.95.



The shopping route passes a temporary image reference to Google Lens through SerpAPI when configured. It exposes merchant, title, price and link information returned by the provider. The prototype presentation describes an experimental best-deal explanation, while the integrated route returns the provider's matching records.

Wardrobe Completion

This feature estimates the usefulness of an addition by the admissible new outfits it enables. It groups owned or selected pieces by category, enumerates bounded combinations and uses BO1 scores to assess compatibility. Candidate additions prioritize useful categories and come with example looks. An account-scoped overlay can include eligible Studio garments.

For candidate c, the engine sums the weights of newly formed complete outfits whose BO1 scores reach the active threshold. It scales that gain by the role weight and divides by a factor that grows with the number of pieces already present in the candidate's category. Mean compatibility breaks ties. After choosing one addition, the engine updates its working wardrobe before ranking the next addition, so each suggestion is evaluated against the current set.

The prototype demonstration shows photo matching and recommendations for adding one, three or five pieces. Integrated tests use controlled catalog fixtures to verify ownership, eligibility, candidate boundaries and look generation. These checks cover the recommendation workflow on its specified inputs.

Project evidence [D1] pp. 12–15; [D6]; [D12].

5 BO3 – Redundancy Detection

BO3 compares garments of the same category using FashionCLIP cosine similarity. It groups connected high-similarity pairs and distinguishes duplicate, confirmation, similar and novel cases. The confirmation band asks the user to decide about a pair. A purchase check compares a proposed addition with the wardrobe.

The wardrobe summary compares counts by category and colour with catalog reference shares using a Poisson-based test. The integrated export also supplies pixel-derived Lab values and a colour distance threshold.

With n garments and connected duplicate groups of sizes s_g, the clutter index is $\text{round}(100 \sum_g (s_g - 1)/n)$. The colour map compares the observed count x with a Poisson mean $\lambda = Np$, where p is the cell's catalog share and N is the number of mapped garments. It flags cells with at least three observations and upper-tail probability $\Pr(X \geq x) < 0.05/C$, where C is the number of cells tested. A pixel-colour discrepancy can route a visually close pair to user confirmation.

Integrated calibration

The target catalog calibration used 100 synthetic duplicate pairs and 4,986 sampled negative pairs. The duplicate threshold was selected by F1, and the similarity threshold used the 99th percentile of negative similarities. The confirmation threshold defines the interval where the user supplies a judgment.

FashionCLIP calibration measure	**Value**
Similarity threshold	0.7002
Confirmation threshold	0.7627
Duplicate threshold	0.8055
Lab colour distance threshold	13.8
F1 on calibration pairs	0.918
Recall on fabricated duplicates	0.95
Same-type pairs in catalog checks	12,512

The 72 BO3 tests passed in the current separate run. They cover the analysis service, API behaviour and donation helpers. The calibrated values belong to the target catalog and pair construction protocol.

Project evidence [D1] pp. 16–20; [D7]; [D12].

6 BO4 – Personalized Style Modeling

Style Profile

The prototype uses an ImageNet-pretrained EfficientNet-B0 with heads for 19 garment types and four broad styles. The team presentation reports macro-F1 of 0.918 for its selected style classifier. The integrated ONNX path accepts user photos or images associated with likes, prepares 224-by-224 inputs and aggregates style probabilities into a profile. The interface presents the dominant style and catalog examples.

Taste Studio and Bubble Breaker

The integrated Taste engine uses 8,189 products, their visual embeddings and style features. It stores likes, dislikes, saves and seen items per account. A save also supplies positive evidence. Discover samples across style groups. Personalized relevance combines similarity to liked items, a penalty for similarity to disliked items and style-profile affinity.

For You ranks unseen items and uses maximal marginal relevance to diversify the returned set. An exploration fraction reserves room for additional candidates. Bubble Breaker requires three likes. Its parameter α balances normalized relevance and surprise, estimated from smoothed frequencies of styles in the user's likes.

The default Taste score uses the mean similarity to the three closest liked items, or all likes if fewer are available, and subtracts 0.6 times the analogous score for dislikes. A style term contributes 0.5 times the dot product between the candidate's style vector and the mean liked style vector adjusted by the disliked style vector. During diversification, maximal marginal relevance subtracts a penalty based on the closest item already selected for the list.

Track	Experiment or implementation
Style classification	Prototype model comparison and ONNX inference in the application.
Preference research	Two-tower sequence and item-content experiment on POG iFashion, including a JEPA pre-training comparison documented in the presentation.
Integrated recommendation	Reaction-driven ranking using the bundled catalog and style features.
HTTP verification	Actual catalog and ONNX execution, photo upload, account-scoped reactions and recommendation responses.

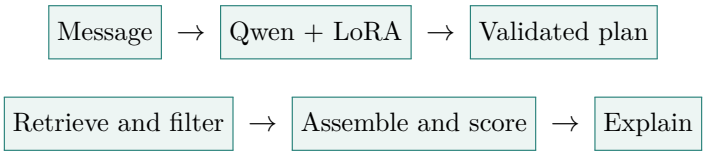
The style macro-F1 refers to the classifier comparison. The application tests exercise the recommendation and state-management paths.

Project evidence [D1] pp. 21–24; [D8]; [D12].

7 BO5 – Explainable Fashion Assistant

Orchestration and request understanding

The prototype notebook defines `FashionCompositionAgent`, a LangGraph orchestrator with four nodes. Its stages understand, compile, execute and explain the request. Conditional transitions send a clarification directly to the explanation stage. The integrated chat coordinates the corresponding operations through the Python class `dressme.engine.Assistant`.



The integrated parser uses Qwen3-1.7B with the exported LoRA adapter. The `Turn` contract represents the action, required and preferred criteria, previous selection references and targeted garment role. Criteria include a field, values, match mode, strength, a source quotation and optional role scope.

Schema and quotation checks turn the generated plan into an executable specification. The count in *two outfits* controls the number of results, while *the second outfit* identifies a saved selection. A follow-up can target one garment role and retain the remaining pieces. The system also keeps pending clarifications when a request needs more information.

The local generation path uses greedy decoding with a ceiling of 1,200 generated tokens. It checks that the encoded prompt and the reserved output budget fit within 4,096 tokens. On the validated GPU profile, the parser runs in NF4 on CUDA. E5 and the BO1 scorer run on CPU. The adapter and tokenizer are loaded once per parser instance and reused across turns.

Supervised data

The exported parser data contain 2,400 training examples, 240 validation examples and 240 test examples. The generator starts from a target plan, constructs a French or English request and adds context where needed. All 2,880 rows carry the origin `synthetic_plan_first`.

Inspection found zero exact message overlaps between split pairs after case folding, Unicode normalization and whitespace normalization. The generated splits represent the same family of request patterns. Composition accounts for 1,734 of the 2,400 training examples. The remaining examples cover product search, replacement, comparison, preference setting, general queries and explanations.

Project evidence [D9], [D10].

QLoRA training methodology

QLoRA propagates gradients through a frozen quantized base model into trainable low-rank adapters. DressMe loads Qwen with 4-bit NF4 and double quantization, uses FP16 compute and stores the trainable LoRA parameters in FP32. The recorded run updated 17,432,576 parameters, approximately 1.003% of the full model with adapters. [1]

Setting	Recorded value
Base model and attention	Qwen3-1.7B, SDPA
LoRA rank, alpha and dropout	16, 32 and 0.05
Attention projections	`q_proj`, `k_proj`, `v_proj`, `o_proj`
Feed-forward projections	`gate_proj`, `up_proj`, `down_proj`
Optimizer and learning rate	AdamW, 10^{-4}
Schedule and regularization	Cosine schedule, 5% warmup, 0.01 weight decay
Batch and accumulation	One example per device, eight accumulation steps
Sequence length and seed	2,048 tokens, seed 42
Training ceiling	Six epochs with gradient checkpointing
Evaluation and checkpointing	Every 50 optimizer updates
Early stopping	Three evaluations, minimum improvement 0.001
Selection criterion	Lowest validation loss

Each example contains instructions, the current request and the target JSON. The loss masks instruction, user and padding tokens. The objective therefore trains the adapter on the structured response. The export records the pinned model revision, adapter hash and tokenizer information so the application can verify what it loads.

Project evidence [D9], [D10].

Optimization and checkpoint selection

The training log records an evaluation every 50 optimizer updates. Each evaluation measures the supervised token loss on the validation split. The best checkpoint was step 400, approximately 1.33 epochs into the run. The early-stopping rule ended training at step 550, approximately 1.83 epochs and 82 minutes after training began.

Checkpoint	Epoch	Validation loss
50	0.17	0.127859
100	0.33	0.059318
150	0.50	0.026613
200	0.67	0.019442
250	0.83	0.017058
300	1.00	0.016808
350	1.17	0.007764
400 (selected)	**1.33**	**0.006507**
450	1.50	0.007235
500	1.67	0.018386
550 (last recorded)	1.83	0.009000

The minimum observed loss determines which adapter is exported. The stopping rule uses a separate improvement threshold of 0.001 and a patience of three evaluations. The selected adapter then generates JSON plans for the structured-output experiments, which measure behaviour at the request level.

Project evidence [D9], [D10].

Parser experiments and results

The paired comparison uses the same 40 validation requests and target plans in the same order. The baseline disables the adapter and the second run enables the selected checkpoint. Both runs score raw generations against the schema and target. A separate seeded sample of 80 held-out requests evaluates the adapted parser.

Metric	**Base, val.** $n = 40$	**LoRA, val.** $n = 40$	**LoRA, test** $n = 80$
Valid output contract	0.00%	100.00%	100.00%
Action accuracy	0.00%	92.50%	75.00%
Context reuse accuracy	0.00%	100.00%	97.50%
Quotation check	0.00%	100.00%	97.50%
Mean strict facet F1	0.00%	95.83%	92.29%
Mean structural facet F1	0.00%	95.83%	95.63%
Exact plan structure	0.00%	92.50%	71.25%

The protocol assigns zero to the metrics when the output contract is unmet. Structural facet F1 compares the represented fields, values, modes, strengths and roles. The strict version also checks quotations. Exact structure evaluates the plan jointly.

In the 80-request test sample, 80 generations satisfied the contract, 60 selected the expected action and 57 matched the expected full plan structure. The measures describe different properties of the same outputs.

The test sample contains 45 composition requests and examples from the other represented actions. These measurements apply to raw parser generations and the synthetic test distribution. Separate scenario replays evaluate the deployed assistant with its schema normalization, retrieval and context rules.

Project evidence [D10], [D11].

Retrieval, assembly and scoring

The RAG stage combines BM25 term matching with normalized multilingual E5 embeddings. It encodes product descriptions as passages and the search input as a query. Each product receives its highest matching passage score. The index is keyed by the encoder revision, product identifiers and passages. The catalog's semantic matrix has 653 rows and 384 dimensions. [2]

Long descriptions are split into windows of up to 360 tokens with 48 tokens of overlap. The query encoder averages its normalized chunks and normalizes the result again. The index preserves passage-to-product alignment so the dense maximum can be computed separately for each product. An offline loader checks the pinned snapshot's weights, tokenizer and pooling configuration before choosing a usable cache.

BO5 ranks retrieval signals by percentiles within each eligible garment role. In hybrid mode, the dense percentile receives weight 0.85 and the lexical percentile receives 0.15. Description-based rules guide suitability for recognized uses such as dinner, sport or an outing. Ownership, garment type, known colour, budget and account context constrain the search where applicable.

The assembler combines eligible items by role, such as top, bottom and shoes. It preserves required or fixed garments and reserves some candidate space for preferences and novelty. An assembly examines up to 20,000 signatures and scores up to 512 candidates. BO1 evaluates the shortlisted combinations. Their ranking score is

$$S = 0.65\,\frac{\mathrm{BO1}}{10} + 0.25\,R + 0.10\,P,$$

where R is mean calibrated retrieval relevance and P is preference evidence. The final selection limits overlap and favours variety relative to the last 12 displayed outfits.

Worked example

The offline replay of *je veux une tenue chic pour diner* returned a white crochet-trimmed bowling shirt, beige pinstripe linen trousers and black leather Derby shoes. Their BO1 score was 9.1424 out of 10. The application reports the components used to rank that exact set.

Component	Value	Weight	Contribution
Normalized BO1	0.914241	0.65	0.594257
Retrieval relevance	0.956766	0.25	0.239191
Preference	0.000000	0.10	0.000000
Total			0.833448

The result combines learned compatibility, retrieval and product-description evidence. The displayed score is reproducible from the recorded components and the submitted product identifiers.

Project evidence [D5], [D9], [D11].

Conversation and personalization

An authenticated account supplies the default gender when the request leaves it open. Explicit request criteria take precedence. Existing accounts can retain an unspecified value. Product groups are mapped by product code from the team's catalog classification, and items in both groups are available under either account filter.

BO5 keeps the current selections, preferences, recent outfits and pending clarifications in account-scoped state.

For a preference-only What-if, the assistant recalculates the two rankings over a common candidate pool. It can expand the pool to cover the preference and records the earlier displayed outfit separately. This permits a comparison of rank changes against a shared set of alternatives.

Replacement contract

The authenticated `/api/assistant/chat` route resolves the current conversation. A request to change the shoes of selection 2 produces a plan with these relevant fields.

```
{"action":"replace", "reuse_previous":true,
 "selection_number":2, "target_role":"shoes"}
```

BO5 fixes the identifiers in the other roles, assembles eligible variants and scores each set through BO1. It updates the targeted slot while preserving the other displayed outfits. The comparison records the target slot, the preserved identifiers and the change in compatibility score.

Scenario evaluation

Suite	Result	Checked behaviour
Conversation replay	15/15	Numbered replacements, retained pieces, preferences across values and score references.
Authenticated replay	12/12	Account defaults, explicit overrides and separate user profiles.
Quantity parsing	10/10	Requested count, recognized occasion and comparison continuity.
Suitability and diversity	22/22	Product descriptions, distinct outfits, repeated requests, targeted replacement and What-if candidate parity.

These runs use the actual Qwen adapter, hybrid RAG and BO1 export. They verify the selected scenarios and share some request patterns. The 22 suitability cases had a median recorded duration of 8.08 seconds on the test machine across different actions and model-loading conditions.

Project evidence [D3], [D9], [D11].

8 BO6 – Smart Wardrobe Digitization

The user submits a garment photo. Image preparation and advisory quality checks precede inference. The BO6 service suggests a category and applicable attributes. The user reviews and confirms the fields, and a template turns the confirmed values into editable description text. The saved record contains the private image reference, confirmed metadata, taxonomy and eligibility information.

The EfficientNet-B0 export uses 11 categories. Transfer learning first trains the classification head for two epochs, then tunes the last backbone blocks for six epochs. Attribute heads use pooled image features and an RGB histogram to estimate sleeves, pattern and colour. The application also supports a separate Garments2Look runtime behind the same service contract and a manual upload path. [3]

From analysis to a saved garment

The `/studio-api/garments/analyze` route creates an account-owned pending upload. The inference client sends the prepared image to BO6, and its category and attribute suggestions populate the review form. The description endpoint uses the confirmed fields. The save operation associates the confirmed garment with the pending upload, and the stored profile can supply a mapped category and stable reference to the wardrobe overlay. BO7 checks the confirmed category and photo type before accepting a garment in a try-on job.

Model comparison

The export records 10,795 training images, 2,310 validation images and 2,336 reserved test images from the catalog apparel subset. The selection rule chooses the lowest CPU latency within 0.01 macro-F1 of the best validation result. The table presents three recorded candidates.

Architecture	Macro-F1	CPU latency	Parameters
ResNet-18	0.8923	30.64 ms	11.18 M
MobileNetV3-Small	0.8768	10.94 ms	1.53 M
EfficientNet-B0	0.8882	28.21 ms	4.02 M

The selected EfficientNet-B0 achieved 93.16% accuracy, 93.22% weighted F1 and 99.22% top-two accuracy. Its 0.70 review threshold gave 95.41% coverage on catalog validation. Every draft still goes through user confirmation. The export's recorded integration evidence includes 17/17 bundle self-tests and a browser upload, review and save journey in the earlier BO6 setup.

Project evidence [D1] pp. 27–30; [D6], [D13], [D14].

9 BO7 – Personalized Virtual Try-On

The user consents to a person input and selects a photo or creates an avatar through Avaturn. The system stores the avatar GLB and renders a 2D image with an arms-down pose. The application resolves selected garments and creates a job under the authenticated account.

The dispatcher and Redis/RQ worker run the configured mock preview or FASHN backend. The real adapter can use a local runtime or remote endpoint according to configuration and availability. In sequential mode, each generated image becomes the person input for the next garment. Private history retains the starting image, garment order and final result. Eligible Outfit of the Day items can enter the same sequence.



Prototype inference experiment

The team presentation records 12 development cases across six person inputs. Each configuration used 30 steps on a Tesla T4. All 36 recorded runs produced an image.

Model	Output size	Runs	Median time
FASHN	576 × 768	12	71.44 s
CatVTON	384 × 512	12	7.46 s
CatVTON	768 × 1024	12	73.50 s

The measurements characterize the individual configurations and their output resolutions. The application suite separately exercises consent, ownership, job state, cancellation, private result handling and the local and remote adapter contracts. The final output is an appearance preview.

Project evidence [D1] pp. 31–34; [D12], [D14].

Job delivery and sequential inference

Job creation validates the selected person input and garment references against the authenticated account. The application records the input snapshot and delivery intent. The caller supplies an idempotency key so a repeated submission can be matched to the same request. The dispatcher publishes queued jobs to Redis/RQ and reconciles deliveries with persisted state.

Stage	Contract
Accept	Persist the validated garment order, person reference and runtime mode. Return the job identifier and status URL.
Dispatch	Publish the delivery for the queued job. Reconcile expired deliveries with the stored attempt history.
Claim	Move the job from queued to running and assign an attempt token. The token identifies the worker attempt allowed to publish the result.
Generate	Check inputs against the stored snapshot. Run one garment at a time and record progress for a sequence.
Publish	Save the final image in private storage and update the job only for the active attempt. Check cancellation before publication.
Retrieve	Resolve the result through the owning account's history and result endpoint.

A sequential job accepts two to four distinct garments in the selected order. If I_0 is the starting person image, G_j is garment j and M is the configured inference backend, the sequence applies $I_j = M(I_{j-1}, G_j)$. Intermediate images are processed in temporary private storage. The final history record retains I_0, the ordered references and the final I_j. The dispatcher allows at most two attempts for eligible reconciled deliveries.

The application tests exercise duplicate submission, result ownership, cancellation and multi-garment ordering through controlled adapters. The notebook timing experiment measures real inference on its recorded development cases.

Project evidence [D12], [D14].

10 Integrated experiments and results

The project uses notebook experiments, export verification, HTTP scenario replays and application tests. The table records the task and denominator for each result so that different kinds of evidence can be interpreted on their own terms.

Experiment	Observed result	Scope
BO1 score parity	3.418087 and 3.318662	Two reference outfits with aligned exported embeddings.
BO1 completion	Recall@1 of 0.532 and 0.323	Category known and category inferred, with re-ranking.
BO2 item workflows	Verified through recorded examples and tests	Photo-match demonstration and controlled wardrobe recommendation checks.
BO3 calibration	F1 0.918	100 fabricated duplicates and 4,986 negatives.
BO4 style comparison	Macro-F1 0.918	Prototype garment style comparison.
BO5 parser test	80/80 valid, 57/80 exact	Held-out synthetic sample of raw outputs.
BO5 suitability replay	22/22 scenarios	Actual Qwen, E5/BM25 and BO1.
BO6 classification	Macro-F1 0.8882	2,310 catalog validation images, EfficientNet-B0.
BO7 inference timing	36/36 images	Three notebook configurations on Tesla T4.
Application tests	367 passed	Current checkout, controlled fixtures and mock try-on as the default.
BO3 tests	72 passed	Separate BO3 test directory.

The offline assistant replay loaded E5 from the project snapshot with downloads disabled and an empty global Hugging Face cache. The dinner request returned a scored outfit from the 653-product catalog. A separate package verification checked model file integrity, catalog alignment and BO1 reference scores.

Project evidence [D1], [D4]–[D8], [D10]–[D14].

11 Scope and conditions of evaluation

Catalog-dependent recommendations. BO1's compatibility and completion results apply to their outfit tasks. BO2 uses the available image and attribute index. BO3 calibration belongs to the target embedding export and the constructed pair distribution. The ranking examples describe outputs for specified inputs and candidate sets.

Style and garment recognition. The BO4 style result refers to the prototype classifier comparison. BO6's numeric results refer to its EfficientNet-B0 catalog validation protocol. The application uses user review as part of the garment digitization workflow.

Conversational evaluation. The parser test samples the synthetic data distribution. Scenario checks provide evidence for particular authenticated and multi-turn behaviours. The assistant's suitability evidence comes from available product attributes and description-based rules, and the ranking applies to a bounded candidate pool.

Try-on and service conditions. BO7's notebook timing depends on the configuration, resolution and hardware. The generated preview represents the model's visual output. Integrated job tests assess orchestration and storage under their stated fixtures. Optional service paths depend on their configured backend and model assets.

12 Reproducing the validation

Use the application lock with Python 3.11 and restore the catalog and model assets relevant to the run. BO6 uses a separate inference environment. The run guide specifies setup and database migrations. Run the tests from the project root with the project Python.

```
python -m pip install -r requirements-integrated.lock.txt
python -m pytest -q tests
python -m pytest -q bo3/tests
```

The recorded application run used `VTON_MODE=mock`, frontend origins matching the test clients, `DRESSME_LLM=off` and `DRESSME_ALLOW_DOWNLOADS=0`. BO5 replay scripts use isolated accounts and state. The portable chat archive provides a manifest for the pinned Qwen, LoRA, E5 and BO1 files.

Project evidence [D2], [D11], [D12].

13 Evidence register

Paths refer to the DressMe repository. The tested implementation is based on commit `35a07a5e` with the local E5 snapshot-loading update. Archived experiments retain their own data and model configuration.

ID	Project evidence
D1	Team presentation, *Dress Me, Phase 3 Prototype*, supplied as `dressmepi.pdf`.
D2	`README.md`, `RUN.md`, `api.py`, `requirements-integrated.lock.txt`.
D3	`docs/SESSION_GENDER_INTEGRATION.md`, `app/auth/`, `alembic/versions/`.
D4	BO1 notebooks under `bo1/`, plus `ootd.py` and `completion.py`.
D5	`dressme/bo1.py`, `bo1/score/config.json`, `diagnostics/verification.json`.
D6	`similar.py`, `wardrobe.py`, `wardrobe_owned.py`, `tests/test_wardrobe_owned.py`.
D7	`bo3/bo3_service.py`, `bo3/calibrate_bo3.py`, `data/bo3_thresholds.json`.
D8	`engine.py`, `reco.py`, `style.py`, `bubble.py`, `tests/test_taste_integration.py`.
D9	BO5 notebook, `dressme/` and `bo5/parser_for_local/data/`.
D10	BO5 `training_receipt.json`, `training_history.csv`, `baseline_validation.json`, `after_validation.json`, `test_report.json`.
D11	`diagnostics/conversation-fixed.json`, `diagnostics/authenticated-chat-replay.json`, `diagnostics/count-parser-live.json`, `diagnostics/relevance-live.json`, local E5 replay and package verification.
D12	`tmp/technical-report-tests.xml`, `tmp/technical-report-bo3-tests.xml`.
D13	BO6 bundle `model_card.md`, `validation_metrics.json`, `selection_report.json` and `docs/PROJECT_STATE.md`.
D14	`app/avatars/`, `app/tryon/`, `tests/test_tryon_real.py`, `tests/test_tryon_remote.py`, `tests/test_tryon_avatar.py`.

References

[1] T. Dettmers, A. Pagnoni, A. Holtzman and L. Zettlemoyer. *QLoRA: Efficient Finetuning of Quantized LLMs*. 2023. arXiv:2305.14314.

[2] L. Wang et al. *Multilingual E5 Text Embeddings: A Technical Report*. 2024. arXiv:2402.05672.

[3] M. Tan and Q. Le. *EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks*. ICML, 2019. PMLR 97, 6105–6114.